SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Dry Run Evaluation

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO1 – Apply (BL3)	**Assessment:**	Assessment Tool 2 – Dry Run Evaluation
Weightage:	35% of CIA	**Duration / Marks:**	60 mins / 100 marks
CO1 Statement:	Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)		
Student Name:	KEERTHANA	**Reg. No.:**	192524489
Section / Batch:	2025-2029	**Date:**	

Code of Conduct

I _KEERTHANA_192524489 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _KEERTHANA

Instructions

- Read each question carefully; answer all four questions (25 marks each, 100 in total).
- Show every dry-run step: the initial queue, stack, priority queue or belief state, each iteration, your assumptions and a clear final answer, and finish with the interpretation asked for.
- Follow the conventions stated in each question (alphabetical expansion, explored set, goal test on removal from the frontier, tie-breaking) and present tables neatly.
- Marks follow the Dry Run Evaluation rubric: understanding of the search problem, state expansion and dry run execution, application of the search algorithm, accuracy of the solution path, and presentation and logical reasoning.
- Upload in GitHub and GCR (question paper with Reg. No., Name, answers, rubrics and evaluation page at the end). Duration: 1 hour (15 minutes per question).

Section / Topic	Focus	Question Nos.
1. Foundations and History of AI	Four approaches to AI, Turing Test, eras and milestones	**Q2**
2. Intelligent Agents: Agents and Environments	Agent, percepts, actions, agent function and program, PEAS	**Q1**
3. The Concept of Rationality	Performance measure, percept sequence, rational vs omniscient	**Q1**
4. The Nature of Environments	Task-environment properties and their classification	**Q1**
5. The Structure of Agents	Simple reflex and model-based reflex agent programs	**Q1**
6. Problem-Solving Agents	Goal formulation, problem formulation, search, execution	**Q2**
7. Problem Formulation	States, initial state, actions, goal test, path cost; belief states	**Q2, Q4**
8. Searching for Solutions	Search graph, frontier, explored set, node expansion	**Q3**

9. Uninformed Search Strategies	Dry runs of BFS, DFS and uniform-cost search	**Q3**
10. Search with Partial Information	Sensorless problems, belief states, percept update	**Q4**

Question 1: Agents, Environments and Rationality – Dry Run of Agent Programs

Topic: Intelligent agents, rationality, nature of environments, structure of agents | 15 minutes

A vacuum-cleaner agent lives in a world with two squares, A and B. At each time step it perceives [its location, status of its own square] and performs one action: Suck, Right, Left or NoOp. The agent starts in square A, both squares are initially Dirty, and dirt never reappears.

Performance measure over T = 6 time steps: +10 for every square cleaned (a Suck on a Dirty square) and −1 for every Left or Right move.

Agent X (simple reflex): if the current square is Dirty, Suck; otherwise move to the other square.

Agent Y (model-based reflex): keeps an internal state of the squares it knows to be clean. If the current square is Dirty, Suck; otherwise, if both squares are known clean, NoOp; otherwise move to the other square.

(a) Write the PEAS description (Performance measure, Environment, Actuators, Sensors) of this agent. **[4 Marks]**

PEAS describes an intelligent agent's task environment in terms of Performance measure, Environment, Actuators and Sensors.

- Performance Measure: +10 for every square cleaned by Suck when it is dirty, and −1 for every Left or Right movement. The objective is to clean both squares while minimizing unnecessary movements.
- Environment: Two squares, A and B, initially dirty. Dirt does not reappear.
- Actuators: Suck, Left, Right and NoOp.
- Sensors: The agent perceives its current location and the cleanliness status of its current square.

(b) Classify the task environment on observability, number of agents, determinism, episodic/sequential, static/dynamic and discrete/continuous, with a one-phrase justification each. **[4 Marks]**

- Observability – Partially observable: The agent observes only its current location and the status of its current square, not the other square directly.
- Number of agents – Single-agent: Only one vacuum agent operates in the environment.
- Determinism – Deterministic: Each action produces a predictable result.
- Episodic/Sequential – Sequential: Actions affect the environment and influence subsequent decisions.
- Static/Dynamic – Static: The dirt does not reappear, and the environment does not change independently while the agent acts.
- Discrete/Continuous – Discrete: Locations, dirt states, actions and time steps are discrete.

(c) Dry-run both agents for all 6 time steps. Complete the table (percept, action, and Y's internal state after the step). **[9 Marks]**

- Initial state: A = Dirty, B = Dirty; agent starts at A.
- Agent X follows a simple reflex rule: if the current square is dirty, it performs Suck; otherwise, it moves to the other square.

- Agent Y follows a model-based reflex rule and remembers which squares are known to be clean. Once both are known to be clean, it performs NoOp.

Step	X: Percept	X: Action	Y: Percept	Y: Action	Y: Known-Clean Squares
1	[A, Dirty]	Suck	[A, Dirty]	Suck	{A}
2	[A, Clean]	Right	[A, Clean]	Right	{A}
3	[B, Dirty]	Suck	[B, Dirty]	Suck	{A, B}
4	[B, Clean]	Left	[B, Clean]	NoOp	{A, B}
5	[A, Clean]	Right	[B, Clean]	NoOp	{A, B}
6	[B, Clean]	Left	[B, Clean]	NoOp	{A, B}

(d) Compute the total performance of X and of Y over the 6 steps. **[3 Marks]**

The performance measure awards +10 for each successful cleaning action and −1 for each movement.

Agent X:

- It cleans two squares, giving (2 \times 10 = 20) points. It makes four movements.
- Total performance = (20 - 4 = \boxed{16})
- **Agent Y:**
- It cleans two squares, giving (2 \times 10 = 20) points. It makes one movement.
- Total performance = (20 - 1 =

\boxed{19}) Therefore, Agent Y performs better

than Agent X.

(e) Interpret: which agent is rational? Both agents receive the identical percept [B, Clean] at step 4 – explain why they act differently, using the structure of the two agent programs, and distinguish a rational agent from an omniscient agent. **[5 Marks]**

- Agent Y is more rational under the given performance measure because it remembers that both squares are clean and avoids unnecessary movements.
- At step 4, both agents receive the same percept, [B, Clean]. However, Agent X moves Left because its simple reflex program considers only the current percept. Agent Y performs NoOp because its internal state records that both squares have been cleaned.
- A **rational agent** selects actions that maximize expected performance using its available percepts and knowledge. An **omniscient agent** would know the actual outcomes of actions in advance. Rationality does not require complete knowledge of the future

Question 2: Foundations and History of AI; Problem-Solving Agent and Problem Formulation

Topic: Foundations and history of AI, problem-solving agents, problem formulation | 15 minutes

Part A. Complete the table by classifying each system or idea under one of the four approaches to AI (thinking humanly, acting humanly, thinking rationally, acting rationally) and naming the era or milestone of AI history to which it belongs.

(a) Complete the table. **[8 Marks]**

System / Idea	Approach to AI	Era / Milestone
Turing Test proposed (1950)	Acting humanly	Early AI; Turing Test milestone
General Problem Solver (Newell and Simon, 1957)	Thinking humanly	Early symbolic AI and cognitive simulation
Aristotle's syllogisms and formal logic	Thinking rationally	Ancient foundations of formal reasoning
Agent that maximizes expected performance	Acting rationally	Rational-agent approach to modern AI

Part B. A problem-solving agent has two jugs: X of capacity 4 litres and Y of capacity 3 litres, both initially empty and without measuring marks. It may fill a jug from a tap, empty a jug on the ground, or pour one jug into the other until the source is empty or the target is full. The goal is to have exactly 2 litres in jug X.

(b) Formulate the problem: state representation, initial state, actions, transition model, goal test and path cost. **[6 Marks]**

1. **State Representation:** Represent a state as ((x,y)), where (x) is the amount of water in jug X and (y) is the amount in jug Y.

2. **Initial State:** ((0,0)), since both jugs are empty.

3. **Actions:** Fill either jug, empty either jug, pour X into Y, or pour Y into X.

4. **Transition Model:** Each action changes the water quantities according to the jug capacities. Pouring continues until the source is empty or the target is full.

5. **Goal Test:** The goal is reached when (x=2), meaning jug X contains exactly 2 litres.

6. **Path Cost:** Each action has a cost of 1. The total path cost is the number of actions performed.

(c) Generate all successor states of (0, 0) and of (4, 0) (leave out actions that leave the state unchanged). **[3 Marks]**

Successors of (0, 0):

- Fill X → ((4,0))
- Fill Y → ((0,3))

Emptying or pouring water produces no change because both jugs are empty.

Successors of (4, 0):

- Empty X → ((0,0))

- Fill Y → ((4,3))
- Pour X into Y → ((1,3))

Pouring Y into X produces no change because Y is empty and X is full.

(d) How many states does the state space contain? Write a shortest solution as a sequence of states and give its path cost. **[3 Marks]**

The possible quantities are (x \in {0,1,2,3,4}) and (y \in {0,1,2,3}).

Therefore, the total number of possible states is:

[
(4+1)(3+1)=5\times4=\boxed{20}
]

A shortest solution is:

Step	Action	State (X, Y)
0	Initial state	(0, 0)
1	Fill Y	(0, 3)
2	Pour Y into X	(3, 0)
3	Fill Y	(3, 3)
4	Pour Y into X until X is full	(4, 2)
5	Empty X	(0, 2)
6	Pour Y into X	(2, 0)

The goal is achieved because jug X contains exactly 2 litres.

Path cost = 6 actions.

(e) Interpret: describe the four phases (goal formulation, problem formulation, search, execution) of a problem-solving agent for this task, and explain why the agent can execute the found solution without looking at new percepts. **[5 Marks]**

1. **Goal Formulation:** Set the goal of obtaining exactly 2 litres in jug X.
2. **Problem Formulation:** Define the states, initial state, actions, transition model, goal test and path cost.
3. **Search:** Find a sequence of actions that reaches the goal, such as the six-action solution above.
4. **Execution:** Perform the actions in the order found.

The agent can execute the plan without checking new percepts at every step because the jug capacities and action outcomes are deterministic and known. Assuming every action succeeds as expected, the planned sequence reaches the goal.

Question 3: Searching for Solutions – Uninformed Search Strategies

Topic: Searching for solutions, BFS, DFS, uniform-cost search | 15 minutes

Consider the weighted undirected graph below. The start node is S and the goal node is G. Use graph search with an explored set; expand neighbours alphabetically, test the goal when a node leaves the frontier, and break ties alphabetically. BFS and DFS ignore edge costs while searching, but path costs are measured with them.

Edge	Cost	Edge	Cost
S – A	1	B – D	1
S – B	4	C – G	6
A – C	2	D – G	2
A – D	5		

(a) Perform BFS from S (queue initially [S]). Do not add a node that is already explored or already in the queue. **[6 Marks]**

BFS uses a queue (FIFO) and explores nodes level by level. The initial queue is [S].

Iteration	Node Dequeued	Queue after Expansion	Explored Set
1	S	[A, B]	{S}
2	A	[B, C, D]	{S, A}
3	B	[C, D]	{S, A, B}
4	C	[D, G]	{S, A, B, C}
5	D	[G]	{S, A, B, C, D}
6	G (Goal)	[]	{S, A, B, C, D}

The goal is found when G is removed from the queue.

BFS Path: S → A → C → G

Path Cost: (1+2+6=\boxed{9})

(b) Perform DFS from S using a stack (initially [S]); push children so that the alphabetically first child is popped first. Write the stack with its top on the left. **[5 Marks]**

DFS uses a stack (LIFO). The stack top is shown on the left. Children are pushed in reverse alphabetical order so that the alphabetically first child is popped first.

Iteration	Node Popped	Stack after Pushing Children
1	S	[A, B]
2	A	[C, D, B]
3	C	[G, D, B]
4	G (Goal)	[D, B]

The goal is found after four nodes are removed from the stack.

DFS Path: S → A → C → G

Path Cost: (1+2+6=\boxed{9})

(c) Perform uniform-cost search from S. Write queue entries as Node:g(parent) and replace an entry whenever a cheaper path to a frontier node is found. **[7 Marks]**

UCS expands the frontier node with the lowest cumulative path cost (g(n)). Entries are written as Node(parent). A frontier entry is updated whenever a cheaper path is found.

Iteration	Node Expanded	g(n)	Priority Queue after Expansion (lowest cost first)
1	S	0	[A:1(S), B:4(S)]
2	A	1	[C:3(A), B:4(S), D:6(A)]
3	C	3	[B:4(S), D:6(A), G:9(C)]
4	B	4	[D:5(B), G:9(C)]
5	D	5	[G:7(D)]
6	G (Goal)	7	[]

At iteration 4, the path through B reduces D's cost from 6 to 5. At iteration 5, the path through D reduces G's cost from 9 to 7.

UCS Path: S → B → D → G

Path Cost: (4+1+2=\boxed{7})

(d) Complete the comparison table. **[4 Marks]**

Strategy	Path Found	Path Cost	Nodes Removed from Frontier	Optimal Here?
BFS	S → A → C → G	9	6	No
DFS	S → A → C → G	9	4	No
UCS	S → B → D → G	7	6	Yes

BFS and DFS find paths based on traversal order rather than cumulative edge cost. UCS finds the least-cost path because it considers the total cost of reaching each node.

(e) Interpret: explain why UCS removed as many nodes as BFS yet found a cheaper path, why DFS needed the fewest nodes here but cannot be trusted in general, and why an explored set is needed in graph search (use iteration 3 of BFS as an example). **[3 Marks]**

- UCS removes six nodes, the same number as BFS, but finds a cheaper path because it expands nodes according to cumulative cost rather than depth. DFS removes only four nodes in this example because it reaches G quickly through A and C; however, DFS is not guaranteed to find the shortest or cheapest path in general.
- An explored set prevents repeatedly expanding nodes and avoids unnecessary cycles. For example, at iteration 3 of BFS, B has D as a neighbour, but D is already in the queue. Therefore, D is not added again.

Question 4: Search with Partial Information – Belief States

Topic: Problem formulation, search with partial information (sensorless and percept-based belief-state search) | 15 minutes

A vacuum agent lives in a two-square world (Left square, Right square), each square being Clean (C) or Dirty (D). Actions: Left, Right and Suck (Left in the left square and Right in the right square leave the agent where it is; Suck cleans the current square). Physical states are written (Agent, Left square, Right square):

State	Description	State	Description
S1	(L, D, D)	S5	(L, C, D)
S2	(R, D, D)	S6	(R, C, D)
S3	(L, D, C)	S7	(L, C, C)
S4	(R, D, C)	S8	(R, C, C)

(a) Initially the agent has no sensors and does not know its location or the dirt status. Formulate the problem over belief states: initial belief state, actions, goal test and path cost. **[3 Marks]**

- **Initial Belief State:** Since the agent has no sensors and does not know its location or the dirt status, the initial belief state contains all eight physical states:

  [
  B_0={S1,S2,S3,S4,S5,S6,S7,S8}
  ]

- **Actions:** Left, Right and Suck.
- **Goal Test:** The goal is achieved when every physical state in the belief state has both squares clean. Thus, every possible state must belong to ({S7,S8}).
- **Path Cost:** Each action has a cost of 1, so the path cost is the total number of actions.

(b) Apply the action sequence [Right, Suck, Left, Suck] and write the belief state after each action. **[6 Marks]**

The agent performs the sequence [Right, Suck, Left, Suck]. Since no sensors are available, all possible resulting states must be retained.

Step	Action	Belief State after Action
0	Start	{S1, S2, S3, S4, S5, S6, S7, S8}
1	Right	{S2, S4, S6, S8}
2	Suck	{S4, S8}
3	Left	{S3, S7}
4	Suck	{S7}

At the final step, the belief state is ({S7}), representing (L, C, C). Both squares are clean, so the goal is achieved.

Total path cost = 4 actions

(c) Now the agent has a local sensor that reports [location, status of the current square]. Starting again from the initial belief state, the first percept is [Left, Dirty]; the agent then performs Suck and perceives [Left, Clean]; it then performs Right and perceives [Right, Dirty]. Fill in the belief state after each action (prediction) and after each percept (update). **[8 Marks]**

The local sensor reports the agent's location and the status of the current square. First, the percept [Left, Dirty] reduces the initial belief state to {S1, S3}.

After each action, the predicted belief state contains all possible resulting states. The percept then removes states inconsistent with the observation.

Step	Action	After Action (Predicted)	Percept	After Percept (Updated)
0	Start	{S1, S2, S3, S4, S5, S6, S7, S8}	[Left, Dirty]	{S1, S3}
1	Suck	{S5, S7}	[Left, Clean]	{S5, S7}
2	Right	{S6, S8}	[Right, Dirty]	{S6}

Explanation:

- Initially, [Left, Dirty] eliminates all states where the agent is at R or the left square is clean.
- After Suck, the left square is clean, leaving S5 and S7 as possible states. Both agree with [Left, Clean].
- After Right, the agent is at R. The percept [Right, Dirty] eliminates S8 because its right square is clean, leaving only S6.

(d) How many different belief states are possible when there are 8 physical states? Why does this make the problem harder than searching over physical states? **[3 Marks]**

- A belief state is any subset of the eight physical states.
- Therefore, the total number of possible belief states is:

  [
  2^8=\boxed{256}
  ]

- This includes the empty set mathematically. If only non-empty belief states are counted, there are (256-1=255).
- Belief-state search is harder than physical-state search because the agent must consider sets of possible states rather than individual states. The number of possible belief states grows exponentially with the number of physical states.

(e) Interpret: explain why the percepts in part (c) shrink the belief state, how this differs from the sensorless case in part (b), and why the sensorless plan can still be called rational. **[5 Marks]**

In part (c), percepts reduce uncertainty by eliminating physical states inconsistent with the sensor observations. For example, [Left, Dirty] reduces the initial belief state from eight states to {S1, S3}.

In the sensorless case in part (b), the agent receives no observations. It must retain every possible state resulting from each action. Nevertheless, the sequence [Right, Suck, Left, Suck] is rational because it guarantees that both squares become clean from every possible initial state.

Thus, **belief-state search enables an agent to reason under uncertainty**, while percept-based updates

allow it to reduce uncertainty when observations are available.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Accuracy of Solution Path	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Q. No. / Criteria	Understa nding of Search Problem	State Expansion and Dry Run Execution	Application of Search Algorithm	Accuracy of Solution Path	Accuracy of Solution Path		Total %
1							
2							
3							
4							
Total							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____